```
PS C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests>  py -3.13 test_optimizer.py

======================================================================
🧪 ЗАПУСК ИНТЕГРАЦИОННОГО ТЕСТА: SGD OPTIMIZER IN VRAM
======================================================================
Тестирование оптимизатора на устройстве: NVIDIA GeForce RTX 2060
C:\Users\Дмитрий\AppData\Local\Programs\Python\Python313\Lib\site-packages\pyopencl\
__init__.py:581: CompilerWarning: Non-empty compiler output encountered. Set the environment
variable PYOPENCL_COMPILER_OUTPUT=1 to see more.
  lambda: self._prg.build(options_bytes, devices),
  ✅ SGD Step Forward: Веса во VRAM обновились со 100% точностью!
  ✅ SGD Zero Grad: Буферы градиентов успешно очищены нулями!

🍻 ТЕСТ ОПТИМИЗАТОРА SGD УСПЕШНО ПРОЙДЕН!
======================================================================


======================================================================
🧪 ЗАПУСК ИНТЕГРАЦИОННОГО ТЕСТА: ADAMW OPTIMIZER IN VRAM
======================================================================
Тестирование AdamW на устройстве: NVIDIA GeForce RTX 2060
Traceback (most recent call last):
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests\test_optimizer.py", line 139, in
<module>
    test_adamw_optimizer()  # <-- Запускаем оба последовательно
    ~~~~~~~~~~~~~~~~~~~~~^^
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests\test_optimizer.py", line 106, in
test_adamw_optimizer
    optimizer_cl.step()
    ~~~~~~~~~~~~~~~~~~^^
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\core\optimizer.py", line 150, in step
    self.program.adamw_step_kernel(
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^
        self.queue, global_size, None,
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    ...<5 lines>...
        np.int32(self.warp_size)
        ^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "<generated: 'pyopencl invoker for 'adamw_step_kernel'-84959cfa190873e8'>", line 7, in
enqueue_knl_adamw_step_kernel
    self._set_arg_multi((0, arg0, 1, arg1, 2, arg2, 3, arg3, 4, arg4, 5, arg5, 6, arg6, 7, arg7, 8, arg8, 9,
arg9, 10, arg10, 11, arg11, 12, arg12,), )

~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
pyopencl._cl.LogicError: clSetKernelArg failed: INVALID_MEM_OBJECT - when processing
arg#1 (1-based):
PS C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests>
```

```python
def test_adamw_optimizer():
    print("\n" + "="*70)
    print("🧪 ЗАПУСК ИНТЕГРАЦИОННОГО ТЕСТА: ADAMW OPTIMIZER IN VRAM")
    print("="*70)

    platforms = cl.get_platforms()[0]
    devices = platforms.get_devices()[0]
    ctx = cl.Context([devices])
    queue = cl.CommandQueue(ctx)
    print(f"Тестирование AdamW на устройстве: {devices.name}")

    # Задаем параметры и случайные веса
    shape = (16, 16)
    np.random.seed(123)
    w_init = np.random.randn(*shape).astype(np.float32)
    g_init = np.random.randn(*shape).astype(np.float32)

    LR = 0.002
    WD = 0.05
    BETAS = (0.9, 0.95)
    EPS = 1e-7

    # 1. Наш Оптимизатор
    W_cl = Tensor.from_numpy(w_init, ctx, queue)
    W_cl.require_grad()
    cl.enqueue_copy(queue, W_cl.grad_buffer, g_init)
    queue.finish()

    optimizer_cl = AdamWOptimizer(
        [W_cl], lr=LR, betas=BETAS, eps=EPS, weight_decay=WD
    )

    # Симулируем 2 шага оптимизации, чтобы проверить накопление истории моментов m и v
    optimizer_cl.step()
    # Загружаем новый случайный градиент для второго шага
    g_init_2 = np.random.randn(*shape).astype(np.float32)
    cl.enqueue_copy(queue, W_cl.grad_buffer, g_init_2)
    queue.finish()
    optimizer_cl.step()

    cl_res_w = HostReturner.to_cpu(W_cl)

    # 2. Эталон в PyTorch
    t_W = torch.tensor(w_init, requires_grad=True)

    # Имитируем шаг 1 в PyTorch
    t_W.grad = torch.tensor(g_init)
    torch_optimizer = torch.optim.AdamW(
        [t_W], lr=LR, betas=BETAS, eps=EPS, weight_decay=WD
    )
    torch_optimizer.step()

    # Имитируем шаг 2 в PyTorch
    torch_optimizer.zero_grad()
    t_W.grad = torch.tensor(g_init_2)
```

```python
    # Имитируем шаг 2 в PyTorch
    torch_optimizer.zero_grad()
    t_W.grad = torch.tensor(g_init_2)
    torch_optimizer.step()

    # Сверяем финальные веса во VRAM с эталоном после двух итераций
    np.testing.assert_allclose(cl_res_w, t_W.detach().numpy(), rtol=1e-4, atol=1e-4)
    print("  ✅ AdamW Step Forward: Моменты и веса обновляются идеально точно!")
    print("\n🎉 ТЕСТ ОПТИМИЗАТОРА ADAMW УСПЕШНО ПРОЙДЕН!")
    print("="*70 + "\n")
```

```python
class AdamWOptimizer:
    def __init__(self, parameters, lr=0.001, betas=(0.9, 0.999), eps=1e-8, weight_decay=0.01):
        """
        Оптимизатор AdamW, выполняющий все расчеты и хранение моментов строго во VRAM.
        """
        self.parameters = list(parameters)
        self.lr = float(lr)
        self.beta1, self.beta2 = float(betas[0]), float(betas[1])
        self.eps = float(eps)
        self.weight_decay = float(weight_decay)

        if not self.parameters:
            raise ValueError("Список параметров для оптимизатора не должен быть пустым.")

        self.ctx = self.parameters[0].ctx
        self.queue = self.parameters[0].queue
        self.step_count = 0  # Счётчик итераций (t) для Bias Correction

        # Узнаем размер варпа
        dummy_code = "__kernel void dummy() {}"
        dummy_prg = cl.Program(self.ctx, dummy_code).build()
        try:
            self.warp_size = dummy_prg.dummy.get_work_group_info(
                cl.kernel_work_group_info.PREFERRED_WORK_GROUP_SIZE_MULTIPLE,
                self.queue.device
            )
        except Exception:
            self.warp_size = 32

        # Загружаем Си-код (он подтянет и SGD, и AdamW)
        kernels_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'cl_kernels'))
        with open(os.path.join(kernels_dir, 'optimizer.cl'), 'r', encoding='utf-8') as f:
            self.program = JITProgramCache.get_or_compile(self.ctx, f.read())

        # ЛЕНИВОЕ ВЫДЕЛЕНИЕ ПАМЯТИ ПОД МОМЕНТЫ (m и v)
        # Каждому тензору параметров сопоставляем два чистых буфера OpenCL той же длины
        self.exp_avg = {}
        self.exp_avg_sq = {}

        for param in self.parameters:
            # Выделяем буферы во VRAM и заполняем их нулями
            m_buf = cl.Buffer(self.ctx, cl.mem_flags.READ_WRITE, param.nbytes)
            v_buf = cl.Buffer(self.ctx, cl.mem_flags.READ_WRITE, param.nbytes)
            cl.enqueue_fill_buffer(self.queue, m_buf, np.float32(0.0), 0, param.nbytes)
            cl.enqueue_fill_buffer(self.queue, v_buf, np.float32(0.0), 0, param.nbytes)

            self.exp_avg[param] = m_buf
            self.exp_avg_sq[param] = v_buf

        self.queue.finish()
```

```python
        self.queue.finish()

    def zero_grad(self):
        """Обнуляет буферы градиентов параметров"""
        for param in self.parameters:
            if param.grad_buffer is not None:
                cl.enqueue_fill_buffer(self.queue, param.grad_buffer, np.float32(0.0), 0, param.nbytes)
        self.queue.finish()

    def step(self):
        """Шаг обновления параметров AdamW"""
        self.step_count += 1

        # Вычисляем Bias Correction на процессоре (один раз на шаг для экономии тактов GPU)
        bias_correction1 = float(1.0 - (self.beta1 ** self.step_count))
        bias_correction2 = float(1.0 - (self.beta2 ** self.step_count))

        for param in self.parameters:
            if param.grad_buffer is None:
                continue

            total_elements = param.size
            total_count = int(np.ceil(total_elements / float(self.warp_size)) * self.warp_size)
            global_size = (total_count // self.warp_size, self.warp_size)

            # Извлекаем скрытые буферы моментов для текущего тензора весов
            m_buf = self.exp_avg[param]
            v_buf = self.exp_avg_sq[param]

            # Вызываем Си-ядро AdamW
            self.program.adamw_step_kernel(
                self.queue, global_size, None,
                param.data_buffer, param.grad_buffer, m_buf, v_buf,
                np.int32(total_elements), np.float32(self.lr),
                np.float32(self.beta1), np.float32(self.beta2),
                np.float32(self.eps), np.float32(self.weight_decay),
                np.float32(bias_correction1), np.float32(bias_correction2),
                np.int32(self.warp_size)
            )

        self.queue.finish()
```

```
__kernel void adamw_step_kernel(
    __global float* W,                  // Буфер весов (data_buffer)
    __global const float* G,            // Буфер градиентов (grad_buffer)
    __global float* M,                  // Буфер первого момента (momentum)
    __global float* V,                  // Буфер второго момента (velocity)
    const int size,
    const float lr,
    const float beta1,
    const float beta2,
    const float epsilon,
    const float weight_decay,
    const float bias_correction1,       // 1.0f - pow(beta1, step)
    const float bias_correction2,       // 1.0f - pow(beta2, step)
    const int warp_size)
{
    int tid = get_global_id(0) * warp_size + get_global_id(1);
    if (tid >= size) return;

    float w_val = W[tid];
    float g_val = G[tid];

    // 1. Отделенный Weight Decay (L2 регуляризация применяется прямо к весу)
    w_val -= lr * weight_decay * w_val;

    // 2. Скользящее среднее моментов градиентов
    float m_val = beta1 * M[tid] + (1.0f - beta1) * g_val;
    float v_val = beta2 * V[tid] + (1.0f - beta2) * g_val * g_val;

    // Перезаписываем состояния моментов во VRAM на месте
    M[tid] = m_val;
    V[tid] = v_val;

    // 3. Коррекция смещения для первых шагов обучения
    float m_hat = m_val / bias_correction1;
    float v_hat = v_val / bias_correction2;

    // 4. Финальное инкрементальное обновление веса
    W[tid] = w_val - (lr / (sqrt(v_hat) + epsilon)) * m_hat;
}
```